PROJECT : Refinery Equipment Failure Prediction (Predictive Maintenance)
Industry Type : Oil Refinery / Petrochemical Plant
Business Problem :
In refineries, critical equipment (pumps, compressors, turbines) fail unexpectedly - Production loss (₹ lakhs/hour) Safety risks High maintenance cost
Business needed: Predict failure before it happens
My Approach (How I solved it)
1. Data Collection
SCADA sensor data:
Temperature
Pressure
Vibration
Flow rate
High-frequency time-series data (per second/minute)
2. Data Preprocessing
Missing value handling (forward fill)
Outlier detection (IQR/Z-score)
Resampling (per minute/hour)
3. Feature Engineering (VERY IMPORTANT)
Lag features → t-1, t-2, t-5
Rolling stats → mean, std
Rate of change (delta)
FFT features (for vibration)
4. Model Building
Used:
XGBoost (best performance)
LSTM (for sequence learning)
Problem type:
Classification (failure vs normal)
5. Deployment
Model deployed using FastAPI
Real-time predictions using:
Kafka (stream ingestion)
Alerts triggered if failure probability > threshold
6. MLOps
Model monitoring (accuracy drop)
Drift detection (sensor distribution change)
Automated retraining pipeline
Impact on Business :
Reduced unplanned downtime by 25% to 30%
Saved maintenance cost by 15% to 20%
Early alerts improved safety
Business Growth :
Increased plant efficiency
Reduced production loss
Better asset utilization

PROJECT : Energy Demand Forecasting & Optimization
Industry: Petrochemical / Refinery

Business Problem :
"Refineries use a lot of energy like electricity, steam, and fuel gas."

Problems:

If we use too much energy - cost increases
If we use too little - production stops

Business wanted:

Predict future energy demand accurately and optimize usage.

My Approach
Step 1: Data Collection

First, I collected different types of data
Past energy usage (important)
Weather data (temperature affects energy)
Production load (more production = more energy)
Shift timing (day/night differences)
Step 2: Data Preparation
Then I cleaned the data:
Filled missing values
Removed outliers
Sorted data by time
Step 3: Feature Engineering (VERY IMPORTANT)
This was the most important step
I created:
Time features:
Hour, Day, Month
Lag features:
Previous values (t-24, t-48)
Rolling mean:
Average of last few hours
External features:
Temperature

I used past energy patterns to predict future energy.

Step 4: Model Building
I tried multiple models:

SARIMA → baseline
Prophet → for seasonality
XGBoost → best performance

Final model = XGBoost

Step 5: Optimization

After prediction, I added optimization logic

Suggested how much energy to use
Avoid peak usage (high cost time)
Balanced load across time

Example:

“Shift heavy energy usage to low-cost hours.”

Step 6: Deployment

I deployed the model in 2 ways:

Batch:
Daily energy forecast
API:
Real-time prediction

Also created dashboard for monitoring

Step 7: MLOps

To maintain model:

Weekly retraining
Drift detection (if pattern changes)
Model versioning using MLflow

Impact on Business :
Reduced energy cost by 15% to 20%
Improved prediction accuracy by 20%
Better planning of energy usage

Business Growth
This helped business in:
Lower operational cost
Better efficiency
Improved sustainability